

Úvod do programovania v jazyku C++

Architektúra jazyka, syntax a základné koncepty, riadenie toku programu, polia a reťazce

Programovanie mechatronických aplikácií
ÚAMT FEI STU

PMEAP Prednáška 1

Prečo C++ pre inžinierov?

- **Výkon a kontrola:** C++ patrí medzi kompilované jazyky s minimálnou abstrakčnou penalizáciou ("Zero-overhead principle"). Poskytuje priamy prístup k pamäti (ukazovatele, manuálna alokácia).
- **Paradigma:** Jazyk podporuje procedurálne, objektovo-orientované (OOP) a generické programovanie.
- **Prenositeľnosť (Portability):** Kód je kompilovaný pre konkrétnu inštrukčnú sadu procesora (x86, ARM), ale samotný zdrojový text je prenositeľný.
- **Aplikácia:** Mikrokontroléry (embedded systémy), real-time systémy, spracovanie signálov, vysoko výkonné výpočty (HPC).

Transformácia zdrojového kódu (textu) na spustiteľné inštrukcie procesora prebieha v štyroch oddelených fázach:

- ❶ **Preprocesor:** Textové náhrady a makrá.
- ❷ **Kompilátor:** Analýza (parsing) a preklad do assembleru / strojového kódu (Object File).
- ❸ **Linker (Zostavovateľ):** Spájanie objektových súborov a knižníc.
- ❹ **Loader:** Zavedenie programu do operačnej pamäte pre exekúciu OS.

Vstup	Výstup (Fáza)
main.cpp	Zdrojový kód (čistý text)
↓	<i>Preprocesor</i> (vyhodnotí <i>#include</i> , <i>#define</i>)
main.i	Expanzný zdrojový kód (interný formát)
↓	<i>Kompilátor</i> (<i>Lexikálna</i> , <i>Syntaktická</i> , <i>Sémantická analýza</i>)
main.o / main.obj	Objektový súbor (strojový kód, nekompletný)
↓	<i>Linker</i> (<i>Pridanie libc, libstdc++, resolving adries</i>)
program.exe / a.out	Spustiteľný binárny súbor (Executable)

Anatómia programu (Hello World)

Listing 1: Základná štruktúra C++ programu

```
// Deklaracia pre procesor: Vloz kniznicu pre vstup/vystup  
#include <iostream>  
  
// Hlavna funkcia, vstupny bod (entry point) programu  
int main() {  
    // Vystup na standardny stream  
    std::cout << "Hello, world!\n";  
  
    // Navratovy kod OS (0 = bez chyby)  
    return 0;  
}
```

- `//`: Jednoriadkový komentár. Ignorovaný kompilátorom. Viacriadkový: `/* ... */`.
- `#include <iostream>`: Direktíva preprocesora. Vloží hlavičkový súbor obsahujúci definície pre `std::cin` a `std::cout`.
- `int main()`: Každý C++ program musí mať funkciu `main`. Vracia celočíselnú hodnotu (`int`) operačnému systému.
- `{ ... }`: Zložené zátvorky definujú *blok (scope)* kódu.
- `;`: Bodkočiarka je **povinná** pre ukončenie každého príkazu (`statement`).

Lexikálne jednotky (Tokens)

Počas fázy *parsing* kompilátor delí text na minimálne sémantické jednotky – **Tokens**.

Existuje 5 základných kategórií:

- ❶ **Kľúčové slová (Keywords):** Rezervované slová (`int`, `return`, `if`, `class`).
- ❷ **Identifikátory:** Užívateľom definované názvy (premenné, funkcie).
- ❸ **Literály:** Konštantné hodnoty zapísané priamo v kóde (`42`, `3.14`, `"Text"`).
- ❹ **Operátory:** Symboly operácií (`+`, `-`, `<<`, `=`).
- ❺ **Interpunkcia:** Štruktúrne znaky (`{`, `}`, `;`, `,`).

Poznámka: Whitespace (medzery, tabulátory, nové riadky) kompilátor ignoruje.

Pravidlá pre identifikátory v C++:

- Môžu obsahovať iba písmená (A-Z, a-z), číslice (0-9) a podčiarkovník (_).
- **Nesmú** začínať číslom (napr. 1variable je syntaktická chyba).
- C++ je **case-sensitive** (Current $\neq$ current).
- Nesmú kolidovať s kľúčovými slovami.

Príklady identifikátorov

```
int voltage_V;           // Validne
double 3phase;           // CHYBA: ZACINA CISLOM
int double;              // CHYBA: KLUCOVE SLOVO
int _internalState;     // Validne, avsak casto rezervovane pre OS
```

Všetky premenné v C++ musia mať definovaný typ (statické typovanie). Typ premennej definuje:

- ❶ **Veľkosť v pamäti:** Koľko bajtov (1 byte = 8 bitov) premenná zaberie.
- ❷ **Rozsah hodnôt:** Aké minimum a maximum dokáže uložiť (ovplyvnené počtom bitov).
- ❸ **Operácie:** Aké inštrukcie procesora sa na daný typ dajú použiť (napr. celočíselné vs. FPU sčítanie).

Celočíselné dátové typy

Pre bežné 32-bit / 64-bit architektúry platia nasledovné štandardné (hoci nie garantované) veľkosti:

Typ	Veľkosť	Modifikátor	Približný Rozsah
char	1 Byte	signed	-128 ... 127
		unsigned	0 ... 255
short	2 Byty	signed	-32 768 ... 32 767
int	4 Byty	signed	$-2.14 \times 10^9 \dots 2.14 \times 10^9$
		unsigned	$0 \dots 4.29 \times 10^9$
long long	8 Bytov	signed	$\pm 9.22 \times 10^{18}$

Typy s pohyblivou rádovou čiarkou a Logický typ

- `float` (4 Byty): Jednoduchá presnosť (IEEE 754). Zhruba 7 platných číslic. Rozsah do 3.4×10^{38} .
- `double` (8 Bytov): Dvojitá presnosť. Zhruba 15 platných číslic. Preferovaný typ pre inžinierske a fyzikálne výpočty.
- `bool` (1 Byte): Reprezentuje pravdivostnú hodnotu. Ukladá sa ako celočíselná hodnota, kde 0 = `false` a akákoľvek nenulová hodnota (štandardne 1) = `true`.

Premenné: Deklarácia a Inicializácia

Premenná mapuje identifikátor na konkrétnu pamäťovú adresu.

Sémantika premenných

```
// 1. Deklaracia (alokacia pamate, hodnota nedefinovana!)  
int current;  
  
// 2. Priradenie hodnoty  
current = 10;  
  
// 3. Deklaracia s inicializaciou (Odporucany sposob)  
double voltage = 230.5;  
  
// Inicializacia viacerych premennych v jednom kroku  
int x = 1, y = 2, z = 3;
```

Výrazy (Expressions) sú kombináciou premenných, literálov a operátorov.

Základné operátory: +, -, *, /, %.

Specifikum: Celociselné delenie

```
int a = 10;
int b = 3;
int result1 = a / b;    // result1 = 3 (truncation - orezanie)
int result2 = a % b;    // result2 = 1 (zvyok, modulo)

// Pre realne delenie musi byt aspon jeden operand float/double
double result3 = 10.0 / 3; // result3 = 3.333...
```

Vstup a Výstup: Streamy (Prúdy)

C++ využíva koncept prúdov (Streams) pre I/O operácie. Dáta prúdia vo forme sekvencie bajtov smerom do (cin) alebo von (cout) z premenných.

```
#include <iostream>
using namespace std;

int main() {
    int value;
    // Operacia vkladania (Insertion operator) <<
    cout << "Zadaj hodnotu: ";

    // Operacia extrakcie (Extraction operator) >>
    cin >> value;

    // Zretazenie operatorov (Chaining)
    cout << "Zadal si: " << value << "\n";
    return 0;
}
```

Formátovanie výstupu: Escape sekvencie

Reťazcové literály uzatvorené v úvodzovkách ("...") môžu obsahovať špeciálne riadiace znaky ASCII tabuľky reprezentované spätným lomítkom (\).

Sekvencia	Reprezentovaný znak (Funkcia)
\n	Nový riadok (Line Feed)
\r	Návrat vozíka (Carriage Return)
\t	Horizontálny tabulátor
\\	Samotný znak spätného lomítka
\"	Znak úvodzovky (zabraňuje predčasnému ukončeniu)
\a	Systémové pípnutie (Alarm/Bell)

Menné priestory (Namespaces)

Menné priestory slúžia na zabránenie kolíziám názvov, ak používate viaceré knižnice, ktoré obsahujú funkcie s rovnakým menom.

Štandardná knižnica C++ umiestňuje svoje symboly do namespace `std`.

Scope resolution (::) vs using directive

```
// Sposob 1: Explicitne použitie scope resolution operatora  
std::cout << "Hello\n";
```

```
// Sposob 2: Globalne otvorenie namespace (v main.cpp, nie v  
hlavicke!)
```

```
using namespace std;  
cout << "Hello\n";
```

Výrazy (Expressions) vs. Příkazy (Statements)

- **Výraz (Expression):** Zápis, po ktorého vyhodnotení procesorom vznikne konkrétna **hodnota**. Napríklad $4 + 2$ má hodnotu 6. Aj "Hello" je výraz s hodnotou pointera na reťazec.
- **Příkaz (Statement):** Ucelená inštrukcia, ktorá niečo **vykoná**. Neprodukuje nutne návratovú hodnotu vo vnútri programu, ale ovplyvňuje stav pamäte. Ukončuje sa bodkočiarkou.

```
int x;           // Deklaracny prikaz
x = 5 + 3;       // Priradzovaci prikaz. (5 + 3) je vyraz.
std::cout << x; // Prikaz volajuci I/O operaciu.
```

Komplexný inžiniersky príklad: Ohmov zákon

```
#include <iostream>

int main() {
    double prud_A, odpor_Ohm; // Identifikatory typu double

    std::cout << "=== Vypocet napatia (Ohmov zakon) ===\n";

    std::cout << "Zadajte prud v Amperoch: ";
    std::cin >> prud_A;

    std::cout << "Zadajte odpor v Ohmoch: ";
    std::cin >> odpor_Ohm;

    // Inicializacia a matematicky vyraz s operatorom nasobenia
    double napatie_V = prud_A * odpor_Ohm;

    std::cout << "Vysledne napatie na svorkach: " << napatie_V << " V\n";

    return 0;
}
```

Chyby v programe sa prejavujú v rôznych fázach životného cyklu:

- **Kompilačné chyby (Compile-time):** Najčastejšie chyby syntaxe. Chýbajúca bodkočiarka, použitie nedeklarovanej premennej, nesúlad typov (Type mismatch).
- **Chyby Linkera (Link-time):** Kód je syntakticky správny, ale Linker nenašiel definíciu volanej funkcie (napr. undefined reference to 'main').
- **Behové chyby (Runtime errors):** Preklad prebehol úspešne, program padne až počas behu. (Napríklad delenie nulou, prístup mimo alokovanú pamäť).
- **Logické chyby:** Program beží plynule, nepadne, ale vracia fyzikálne alebo matematicky nesprávny výsledok.

Motivácia: Prečo riadiť tok programu?

- **Sekvenčné vykonávanie:** Normálne program vykonáva príkazy postupne od prvého po posledný.
- **Limitácie sekvencie:** Program by nebol veľmi užitočný, ak by zakaždým vykonal presne tú istú postupnosť inštrukcií bez ohľadu na vstupy.
- **Dynamické správanie:** Potrebujeme meniť, ktoré príkazy sa spustia a kedy, v závislosti od okolností (napr. vstup zo súboru, reakcia na hráča v hre).
- **Riadiace štruktúry (Control structures):** Časti kódu obsahujúce príkazy, ktoré definujú exekúciu na základe stavu programu.
- Typicky sa delia na dve hlavné kategórie: **podmienky** (conditionals) a **cykly** (loops).

Relačné operátory v C++

Podmienky umožňujú testovať vstupy pomocou operátorov. Relačné operátory testujú vzťah medzi dvoma výrazmi. Výsledkom je vždy logická hodnota (true alebo false).

Operátor	Význam
>	Väčší ako
>=	Väčší alebo rovný
<	Menší ako
<=	Menší alebo rovný
==	Rovná sa (test rovnosti, nie priradenie!)
!=	Nerovná sa

Inžiniersky detail: Tieto operátory sa prekladajú do inštrukcií na porovnanie v procesore (napr. CMP v x86 architektúre), po ktorých nasledujú inštrukcie podmieneného skoku (napr. JNE, JG).

Logické operátory

Na spájanie jednoduchých relačných výrazov do komplexnejších Boolean výrazov používame logické operátory:

Operátor	Význam	Typ operátora
&&	Logické AND (a zároveň)	Binárny
	Logické OR (alebo)	Binárny
!	Logické NOT (negácia)	Unárny

Tabuľka pravdivostných hodnôt (AND a OR):

a	b	a && b	a b
true	true	true	true
true	false	false	true
false	true	false	true
false	false	false	false

Sémantika hodnôt: Čo je pravda?

V C/C++ má koncept logickej hodnoty zaujímavú črtu založenú na celých číslach:

- Hodnota `false` je reprezentovaná číslom 0.
- **Čokoľvek, čo nie je 0, je `true`.**

Dôsledky v praxi:

- Reťazec `"Hello, world!"` vyhodnotí prekladač ako `true` (ide o nenulový smerník).
- Číslo 2 je `true`.
- Akákoľvek celočíselná premenná (`int`) obsahujúca nenulovú hodnotu je `true`.
- `!x` vráti `false`, ak `x` je napríklad 6.

Podmienené vetvenie: if a else

Konstrukt `if` spúšťa blok príkazov, ak podmienka vráti `true`. `if-else` umožňuje voľbu medzi dvoma blokmi (mutually exclusive).

Základný if a if-else

```
1 if (podmienka) {  
2     prikazA1;  
3     prikazA2;  
4 } else {  
5     prikazB1;  
6 }
```

Reťazenie pomocou else if

```
1 if (podmienka1) {  
2     prikazA;  
3 } else if (podmienka2) {  
4     prikazB;  
5 } else {  
6     prikazC;  
7 }
```

- V bloku `if-else-if` sa vykoná **bud' jeden alebo žiadny blok**.
- Ak blok obsahuje len jeden príkaz, zložené zátvorky `{}` môžu byť vynechané. V softvérovom inžinierstve sa to však neodporúča kvôli prehľadnosti a bezpečnosti.

Praktický inžiniersky príklad: Vetvenie if-else

Ukážka aplikácie relačných operátorov v štruktúre if-else-if:

```
#include <iostream>
using namespace std;

int main() {
    int x = 6;
    int y = 2;

    if (x > y)
        cout << "x is greater than y\n";
    else if (y > x)
        cout << "y is greater than x\n";
    else
        cout << "x and y are equal\n";

    return 0;
}
```

Architektúra vetvenia: switch-case

switch-case je štruktúra, ktorá porovnáva výraz s konštantami. Prekladače ju často optimalizujú pomocou skokových tabuliek (jump tables) pre časovú zložitosť $\mathcal{O}(1)$.

Syntaktická štruktúra switch-case

```
switch (vyraz) {  
    case konstanta1:  
        prikazA1;  
        break;  
    case konstanta2:  
        prikazB1;  
        break;  
    default:  
        prikazZ1;  
}
```

Ak sa nájde zhoda, príkazy pod case sa vykonávajú, až kým nenarazia na kľúčové slovo break. Zložené zátvorky pre vnútorné príkazy nie sú potrebné.

Praktický príklad a "Fall-through" efekt

Špecifické správanie switch-case: Ak chýba break, vykonávanie plynulo "prepadne" (fall-through) do nasledujúceho bloku case.

```
switch(x) {  
    case 1:  
        cout << "x is 1\n";  
        break;  
    case 2:  
    case 3:  
        cout << "x is 2 or 3\n";  
        break;  
    default:  
        cout << "x is not 1, 2, or 3\n";  
}
```

Ak $x = 2$, vykoná sa porovnanie case 2. Keďže tam chýba kód a break, tok prejde do case 3 a vypíše "x is 2 or 3".

Riadenie opakovania: Cykly (Loops)

- Kým podmienky vykonávajú kódy *ak* je podmienka splnená, cykly vykonávajú kódy *pokým* (zatiaľ čo) podmienka pretrváva.
- Jazyk C++ ponúka tri základné typy cyklov:
 - 1 `while` (podmienka sa testuje pred vstupom do bloku)
 - 2 `do-while` (podmienka sa testuje na konci bloku)
 - 3 `for` (iteračný cyklus s integrovanou správou premennej)

Cykly s podmienkou na začiatku: while

Cyklus `while` neustále opakuje blok príkazov, kým je podmienka vyhodnotená ako `true`.

```
int x = 0;
while (x < 10) {
    x = x + 1;
}
cout << "x is " << x << "\n";
```

- Podmienka sa overí predtým, ako procesor vstúpi do tela cyklu.
- Ak podmienka hneď na začiatku neplatí, telo cyklu sa nevykoná **ani raz**.
- Tento program iteruje 10-krát a po ukončení vypíše "x is 10".

Cykly s garanciou exekúcie: do-while

Cyklus do-while zabezpečuje (garantuje), že blok príkazov bude vykonaný **aspoň jedenkrát**.

```
do {  
    statement1;  
    statement2;  
} while (condition);
```

- Procesor vstúpi do bloku do, vykoná všetky inštrukcie a až následne vyhodnotí podmienku vo while.
- Ak je splnená, vráti sa na začiatok bloku.
- **Dôležitý syntaxový detail:** Všimnite si bodkočiarku po zátvorkách príkazu while(condition); – tá je povinná!

Kompaktný iteračný cyklus: for

Slučka `for` združuje inicializáciu, podmienku a inkrementáciu na jedno miesto. Typicky sa využíva, keď je počet iterácií známy vopred.

```
for (inicializacia; podmienka; inkrementacia) {  
    prikaz;  
}
```

// Prakticky priklad:

```
for (int x = 0; x < 10; x = x + 1)  
    cout << x << "\n";
```

Ak je riadiaca (counter) premenná zadefinovaná pred cyklom, prvá časť môže ostať prázdna: `for(; x < 10; x = x + 1)`. Prvá bodkočiarka je však v zátvorkách stále povinná. Tento program vytlačí hodnoty 0 až 9, každú na nový riadok.

Ekvivalencia kompilácie: for a while

Z inžinierskeho hľadiska na úrovni kompilátora (napr. Clang IR alebo GCC GIMPLE) je cyklus `for` čistým syntaktickým cukrom (syntactic sugar) a prekladá sa rovnako ako modifikovaný `while`.

Zápis FOR

```
for (int x = 0; x < 10; x = x +  
    1) {  
    cout << x << "\n";  
}
```

Ekvivalentný WHILE

```
int x = 0;  
while (x < 10) {  
    cout << x << "\n";  
    x = x + 1;  
}
```

Inicializácia prebieha pred testom `while`, pričom inkrementácia sa technicky vykoná až na samom konci tela `while` cyklu. Je dobrou praxou nechať inkrementáciu ako posledný krok.

Vnorené radiace štruktúry (Nested Structures)

Rovnako ako v iných jazykoch, radiace štruktúry môžeme ľubovoľne vnárať do seba (napr. cykly do cyklov, podmienky do cyklov).

```
for (int x = 0; x < 4; x = x + 1) {  
    for (int y = 0; y < 4; y = y + 1) {  
        cout << y;  
    }  
    cout << "\n";  
}
```

- Výstup tohto programu vytlačí štyri riadky reťazca 0123.
- **Technická poznámka:** Hlboké vnáranie podmienok (deep nesting) môže degradovať výkon na procesoroch kvôli chybám v predikcii vetvenia (Branch Prediction Misses) a sťažuje čitateľnosť zdrojového kódu. Dvojitý vnorený cyklus iterujúci N -krát generuje asymetrickú časovú zložitosť $\mathcal{O}(N^2)$.

Definícia a štruktúra poľa (Array)

- **Polia (Arrays):** Umožňujú uložiť viacero hodnôt spoločne ako jednu logickú jednotku.
- C/C++ definuje pole ako **pevný počet prvkov rovnakého dátového typu**.
- Prvky sú uložené **sekvenčne (za sebou) v pamäti**.
- Veľkosť poľa sa označuje ako jeho **dimenzia** (dimension).

Syntaktický vzor deklarácie:

```
typ nazovPola [dimenzia];  
  
// Príklad: Celociselné pole so štyrmi prvkami  
int arr[4];
```

- Polia v C++ sú **indexované od nuly (zero-indexed)**.
- Prvý prvok má vždy index 0.
- Prístup k tretiemu prvku poľa sa zapíše ako `arr[2]`.
- Zadaný index musí byť kladné celé číslo v intervale od 0 do $n - 1$, kde n je dimenzia poľa.
- Index môže byť konštanta, premenná alebo vypočítaný výraz:

```
arr[5];           // Priamy prístup  
arr[i];           // Prístup pomocou premennej  
arr[i+3];         // Prístup na základe vypočítaného výrazu
```

Metódy inicializácie polí

Rovnako ako bežné premenné, aj prvky poľa musia byť pred použitím inicializované, inak získame neočakávané (náhodné) výsledky z pamäte.

1. Dodatočná inicializácia prvok po prvku:

```
int arr[4];  
arr[0] = 6; arr[1] = 0; arr[2] = 9; arr[3] = 6;
```

2. Inicializácia počas deklarácie pomocou zoznamu:

```
int arr[4] = {6, 0, 9, 6};
```

3. Automatické odvodenie veľkosti kompilátorom:

```
// Kompilátor automaticky vytvorí pole s dimenziou 8  
int arr[] = {6, 0, 9, 6, 2, 0, 1, 1};
```

Praktický príklad: Dynamický vstup a výstup

Polia možno jednoducho napĺňať a čítať hodnotami, ktoré vopred nepoznáme, s využitím cyklov (for slučka).

```
#include <iostream>
using namespace std;

int main() {
    int arr[4];
    cout << "Please enter 4 integers:" << endl;
    for(int i = 0; i < 4; i++)
        cin >> arr[i]; // Nacitanie do pola
    cout << "Values in array are now:";
    for(int i = 0; i < 4; i++)
        cout << " " << arr[i]; // Vypis hodnot
    cout << endl;
    return 0;
}
```

Polia ako argumenty funkcií

Polia môžeme posilať do funkcií ako argumenty. Pri deklarácii funkcie uvedieme parameter ako pole **bez zadania dimenzie**.

```
// Vsimnite si, ze specifikujeme aj dlzku (length) pola
int sum(const int array[], const int length) {
    long sum = 0;
    for(int i = 0; i < length; sum += array[i++]);
    return sum;
}

// Volanie v main():
int arr[] = {1, 2, 3, 4, 5, 6, 7};
cout << "Sum: " << sum(arr, 7) << endl;
```

Kritický technický detail

V jazyku C/C++ sa polia do funkcií **odovzdávajú odkazom (passed by reference)**. Kód vo funkcii manipuluje priamo s pôvodným pamäťovým priestorom, nie s jeho kópiou.

- Technicky sa odovzdáva smerník (pointer) na prvý prvok poľa.
- **Dôsledok:** Akékoľvek zmeny, ktoré vykonáte na poli vo vnútri funkcie, **sa prejaví aj v pôvodnom volajúcom prostredí (calling scope)**.
- Aj preto pole z predchádzajúceho príkladu použilo kvalifikátor `const int array[]`, čím funkcii explicitne zakazujeme pole modifikovať.

Viacrozmerné polia (Multidimensional Arrays)

C++ plne podporuje viacrozmerné polia (matice, tenzory) jednoduchým pridaním ďalších množín hranatých zátvoriek.

- Deklarácia dvojrozmerného poľa (matice):
`typ nazovPola[dimenzia1][dimenzia2];`
- Obsahuje presne `dimenzia1` x `dimenzia2` prvkov rovnakého typu.
- Možno si ho predstaviť ako **pole polí**.
- Prvý index určuje, ku ktorému z `dimenzia1` podpolí sa pristupuje, druhý index pristupuje k prvku v danom podpoli.

```
int twoDimArray[2][4];  
twoDimArray[0][0] = 6;  
twoDimArray[1][3] = 1;
```

Inicializácia viacrozmerných polí

Podobne ako 1D polia, aj viacrozmerné môžeme inicializovať pri deklarácii. Prvky sa ukladajú sekvenčne riadok po riadku (row-major order).

Plochá inicializácia:

```
int twoDimArray[2][4] = {6, 0, 9, 6, 2, 0, 1, 1};
```

Hierarchická inicializácia (pre lepšiu čitateľnosť):

```
int twoDimArray[2][4] = { {6, 0, 9, 6}, {2, 0, 1, 1} };
```

Poznámka: Pri viacrozmerných poliach musia byť pri inicializácii **vždy definované rozmery**, inak kompilátor nedokáže určiť správne rozdelenie (partitioning) prvkov.

Pamäťová abstrakcia viacrozmerých polí

Je dôležité pochopiť vzťah medzi viacrozmerným poľom v kóde a jeho fyzickou prítomnosťou v RAM procesora.

Architektúra pamäte

Viacrozmerne polia sú len abstrakciou pre programátorov. Fyzicky sú všetky prvky poľa v pamäti uložené **sekvenčne (lineárne za sebou)**.

Nasledujúce deklarácie alokujú v pamäti identický blok s rovnakou štruktúrou posunov (offsets):

```
int arr[2][4];    // Abstrakcia 2x4 matice
// ...je pamatovo uplne rovnaka vec ako...
int arr[8];       // Linearne pole s 8 prvkami
```

Reťazce znakov (Strings)

- Textové literály v C++ (napr. "Hello, world!") sú kompilátorom reprezentované ako **sekvencia znakov v pamäti**.
- Technicky povedané, klasický C-reťazec je jednoducho **znakové pole (character array)** a dá sa tak s ním manipulovať.

Pre manipuláciu so štandardnými knižnicami z jazyka C integrujeme nasledujúce knižnice

	Knižnica v C++ (C hlavička)	Funkcionalita
(hlavičky):	<cctype> (ctype.h)	Operácie so znakmi (character handling)
	<cstdio> (stdio.h)	Vstupné a výstupné operácie (I/O)
	<cstdlib> (stdlib.h)	Všeobecné utility a konverzie
	<cstring> (string.h)	Operácie a manipulácia s reťazcami

Ukončovací Null znak (Null terminator)

Reťazce v štandarde C vyžadujú prítomnosť špeciálneho ukončovacieho znaku.

```
// Rucne vyskladany retazec znak po znaku:  
char helloworld[] = {'H','e','l','l','o',' ',' ','w','o','r','l','d',  
                    ',','!','\0'};
```

- V poli sa nachádza špeciálny znak `'\0'`, známy ako **null znak (null character)**.
- Tento znak slúži ako indikátor konca reťazca pre funkcie pracujúce s pamäťou.
- Ak inicializujeme pole priamo stringovým literálom, null znak nemusíme písať, **kompilátor ho vloží automaticky**:

```
char helloworld[] = "Hello, world!";
```

Spracovanie znakov: Knihnica `<cctype>`

Knihnica `<cctype>` ponúka funkcie pre prácu a detekciu vlastností jednotlivých znakov. Tieto funkcie zvyčajne vracajú Boolean hodnotu (true/false).

- `isalpha(char)`: Overuje, či je znak písmenom abecedy.
- `isupper(char)`: Overuje, či ide o veľké písmeno.
- `ispunct(char)`: Overuje, či ide o interpunkčné znamienko.
- `tolower(char)`: Skonvertuje znak na malé písmeno.

Ukážka filtrácie textu pomocou `cctype`:

```
char current = messyString[0];
for(int i = 0; current != '\0'; current = messyString[++i]) {
    if (isalpha(current))
        cout << (char)(isupper(current) ? tolower(current) : current
        );
}
```

Manipulácia s reťazcami: Knižnica `<cstring>`

Pre operácie ako kopírovanie blokov textu alebo zlučovanie (konkatenáciu) slúži knižnica `<cstring>`. Zodpovednosť za dĺžku pamäte a buffer overflow je však vždy na programátorovi!

```
#include <iostream>
#include <cstring>
using namespace std;
int main() {
    char fragment1[] = "I'm a s";
    char fragment2[] = "tring!";
    char fragment3[20];
    char finalString[20] = "";
    strcpy(fragment3, fragment1);           // Skopiruje fragment1 do
        fragment3
    strcat(finalString, fragment3);         // Pripne fragment3 na koniec
        finalString
    strcat(finalString, fragment2);         // Pripne fragment2 na koniec
        finalString
    cout << finalString; // Vypise: I'm a string!
```

- C++ je vysoko výkonný kompilovaný jazyk, ktorý je spracovávaný v 3 krokoch (Preprocesor, Kompilátor, Linker).
- Každý program musí začínať blokom vo vnútri `int main()`.
- Syntax je extrémne striktná (case-sensitive, vyžaduje `;`).
- Pamäť je štruktúrovaná prísne cez staticky typované premenné (`int`, `double`, `bool`).
- I/O operácie fungujú formou smerovaných streamov (`std::cout <<`, `std::cin >>`).

- Pole je sekvencia hodnôt v pamäti fixnej dĺžky prístupná pomocou indexu od 0 do $n - 1$.
- Polia sú referenčnými typmi pri odovzdávaní do funkcií (manipulujeme s originálnou pamäťou).
- Viacrozmerné polia sú v pamäti len plochá abstrakcia.
- C-reťazec je char pole ukončené špeciálnym null znakom `'\0'`.
- Odporúča sa neustále čítať oficiálnu technickú dokumentáciu pre štandardné C/C++ knižnice pre pochopenie presnej sémantiky funkcií.

Otázky a diskusia